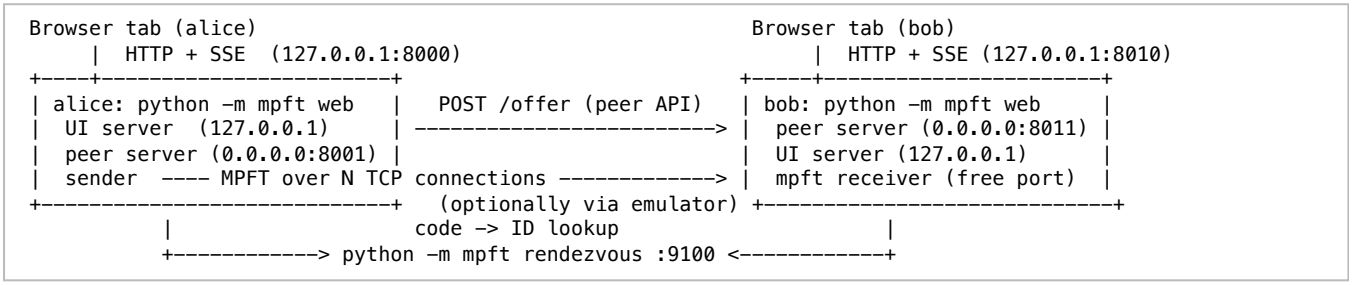
Web interface

A browser front end for sending and receiving files, with a live view of what the protocol is doing. The data plane is unchanged: the sender app calls `mpft.sender.run_transfer`, the receiver app runs the ordinary `mpft.receiver`, so Wireshark still sees plain MPFT.

Architecture

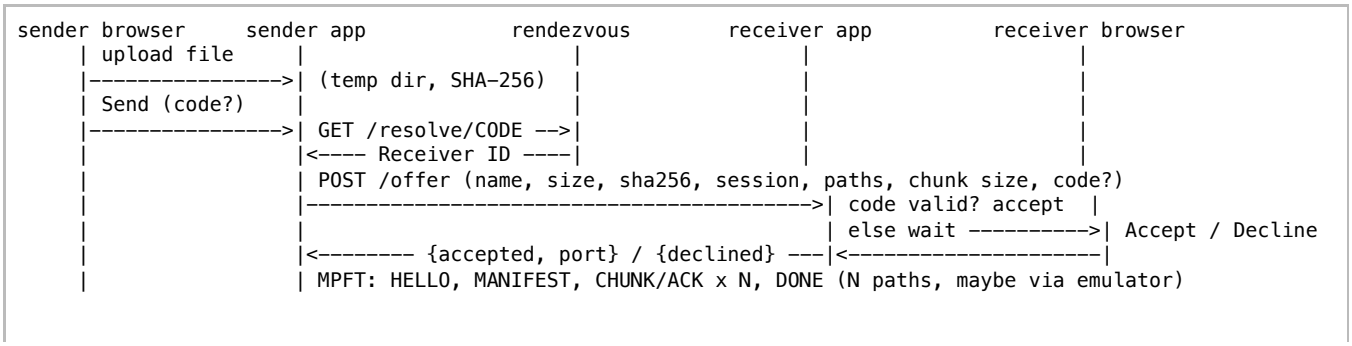


Piece	File	Job
Event bus	`mpft/events.py`	Thread-safe; sender, receiver and emulator emit, the web layer subscribes
Node	`web/node.py`	Identity, one-time code, offers, received files, history
Peer API	`web/peer.py`	Only `GET /health` and `POST /offer`, bound to 0.0.0.0
UI server	`web/ui.py`, `web/feed.py`	Pages, JSON API, SSE stream, downloads. Bound to 127.0.0.1
Send job	`web/sendjob.py`	Resolve target, offer, then `run_transfer`
Rendezvous	`web/rendezvous.py`	Code to Receiver ID lookup
Profiles	`web/netprofiles.py`	Emulator paths for None, Asymmetric, Lossy, Path failure

Receiver ID. `base32("host:peer_port")` shown in groups of four letters. It decodes locally, so no server is needed. The host is the Tailscale 100.x address if there is one, else the LAN IP, else 127.0.0.1 (`--host` overrides).

One-time code. Six characters without confusable letters, valid 10 minutes, single use. The receiver registers `code -> ID` with the rendezvous service; the sender resolves it. A valid code makes the receiver accept without a click.

Offer flow



```
    |                |------------------------------------------>| verify, rename       |
    |<-- SSE events --|                                          |--- SSE events ------>|
```

Offers without a valid code wait up to 120 s for Accept or Decline. Received files are staged in `<out>/.incoming/<id>/`, then renamed into `<out>/` with a numeric suffix if the name exists (never overwritten).

Run on one Mac (two instances, two tabs)

```
source .venv/bin/activate
python -m mpft rendezvous --host 127.0.0.1 --port 9100                          # terminal 1
python -m mpft web --name alice --ui-port 8000 --peer-port 8001 --out ./alice-dl --host 127.0.0.1 --rendezvous
python -m mpft web --name bob   --ui-port 8010 --peer-port 8011 --out ./bob-dl   --host 127.0.0.1 --rendezvous
```

Open `http://127.0.0.1:8010/#/receive` (bob) and `http://127.0.0.1:8000/#/send` (alice). Copy bob's ID or code into alice's Send page, choose a file and press Send. With an ID, press Accept on bob's page; with a code there is no click.

Run on two Macs over Tailscale

1. Install Tailscale on both Macs and sign in to the same tailnet; note each 100.x address (`tailscale ip -4`).
2. On the Mac that will host lookups, start `python -m mpft rendezvous --port 9100`. **The rendezvous URL is that Mac's 100.x address:** `http://100.x.y.z:9100`.
3. On each Mac: `python -m mpft web --name NAME --ui-port 8000 --peer-port 8001 --out ./downloads --rendezvous http://100.x.y.z:9100`. The ID then contains the Tailscale address automatically. The URL can also be set in Receive > Settings.
4. Allow incoming connections if macOS asks (peer port 8001 and the transfer ports).
5. Sender enters the receiver's ID (or code) in the browser at `http://127.0.0.1:8000`.

Security: MPFT has no built-in encryption and the rendezvous service has no authentication. Tailscale (WireGuard) encrypts traffic between the Macs; do not expose the peer port or the rendezvous port to the open internet. The UI server only listens on 127.0.0.1 and rejects foreign Host or Origin headers, because it controls the machine.

Network conditions

The Send page offers None, Asymmetric (paths progressively slower), Lossy (about 5 % of chunks damaged) and Path failure (last path dies 3 s after it connects; use a file of 50 MB or more). The emulator proxies run inside the sender app, in front of the receiver. The Transfer page then shows Kill path N and Set corruption (percent of chunks; converted to the emulator's per-16 KiB-block probability). `python -m mpft emulate --control-port 9300` exposes the same control over HTTP: `GET /state`, `POST /kill/N`, `POST /corrupt/RATE`.

Wireshark for a web transfer

Ports are chosen per transfer, so the Transfer page prints the exact command and filter. Re-copy `wireshark/mpft.lua` into the Wireshark plugin folder: it now also recognises MPFT by its header on any port.

```
sudo tcpdump -i lo0 -w mpft.pcap 'tcp port 59216 or tcp port 59217'   # as shown on the Transfer page
```

Display filters: `mpft.type == 4` (chunks), `mpft.type == 2` (manifest), `mpft.type == 6` (NACKs).

Limits and honest notes

- The offer's size and SHA-256 are what the receiver shows; the data plane verifies against the manifest the sender sends.
- Corruption in the UI is emulated on the sender side of the emulator; real packet loss is not emulated (TCP hides it).
- The event feed keeps the last 100 000 events per process, so a page reload replays a transfer; very large transfers with tiny chunks lose their oldest events.

Troubleshooting

Symptom	Cause and fix
"Could not reach the receiver"	Wrong ID, receiver app not running, or firewall. Open `http://RECEIVER:PEERPORT/health` from the sender Mac.
"No rendezvous URL is set"	Add it in Receive > Settings, or start with `--rendezvous`; or send by ID.
"Code not found or expired"	Codes live 10 minutes and work once. Press New code on the receiver.
"Too many wrong codes"	Five failures per minute per client are allowed; wait a minute.
Port already in use	A previous instance is still running: `lsof -i :8001`.
ID shows 127.0.0.1 across Macs	No Tailscale or LAN address found; start with `--host 100.x.y.z`.
Packets look like plain TCP in Wireshark	Old copy of `mpft.lua`; reload the new one.